



PYTPRA

Maschinelles Lernen in der Praxis

Scikit

Lernen

- Typischerweise beobachten wir Paare von Zahlen, etwa Stundenlohn (wage) und Jahre an Berufserfahrung (experience)
- Daraus wollen wir **lernen**, wie der Zusammenhang zwischen beiden Grössen im Wesentlichen ist
- D.h. uns ist klar, dass der Zusammenhang eher im Mittel als in jedem einzelnen Fall gilt
- In Formeln ausgedrückt gibt es einen datengenerierenden Prozess, der hier folgende Gleichung sei
Das **wahre** Modell (w) sei: $y = a + bx + e$
- Der Term e heisst Störterm und fasst alles zusammen, was sonst noch eine Rolle spielt, aber nicht leicht beobachtbar ist. Unsere Beobachtungen können wir nutzen, um eine Schätzung für (w) vorzunehmen
Das **geschätzte** Modell (g) sei: $y = \alpha + \beta x + u$
- Den Vorhersagefehler u nennen wir Residuum
- Wir haben n Beobachtungen und indizieren diese mit $i = 1, \dots, n$.

Lernverfahren

- Der Term e heisst Störterm und fasst alles zusammen, was sonst noch eine Rolle spielt, aber nicht leicht beobachtbar ist. Unsere Beobachtungen können wir nutzen, um eine Schätzung für (w) vorzunehmen

Das **geschätzte** Modell (g) sei: $y = \alpha + \beta x + u$

- Den Vorhersagefehler u nennen wir Residuum
- Ein Verfahren, die beiden Modellparameter zu bestimmen, ist die Methode der kleinsten Quadrate, d.h. wir minimieren die Summe der Fehlerquadrate u^2 durch Wahl von α und β
- Zum Glück führt Python die Berechnung der „**Kleinsten Quadrate**“ (**KQ**) für uns aus
- Wir wollen dazu ein Laborexperiment mit dem Jupyter Notebook **PYTPRA_01_01_ScikitLinReg.ipynb** durchführen

Tafel: Gerade durch
Punktwolke

Experiment

- Auch wenn der Datensatz bereits Stundenlöhne enthält, wollen wir diese zunächst ignorieren
- **Welche Stellen im Code sind unklar?**
- **Wieviele Frauen befinden sich am Anfang des Datensatzes?**
- **Hinweis: ggf. in TargetDirectory Setzung Doppelung des Backslash nötig, d.h. \ statt **

```
# Import required packages
import pandas as pd
import numpy as np

#Scikit
from sklearn.linear_model import LinearRegression #for lin reg
from sklearn.metrics import r2_score

import statsmodels.api as sm

import os

targetDirectory = "C:\Data\Dynamic\LECTURES\Bachelor\ML in der Praxis\Modelle"
os.chdir(targetDirectory)
print("Working directory in use: ", os.getcwd())
```

Working directory in use: C:\Data\Dynamic\LECTURES\Bachelor\ML in der Praxis\Modelle

```
### data is looked for in wd
df_WAGES = pd.read_csv('wages.txt', sep = "\t") #txt is tab separated, data from Verbeek,
df_WAGES.head()
```

	EXPER	MALE	SCHOOL	WAGE
0	9	0	13	6.315296
1	12	0	12	5.479770

Experiment

- Stattdessen erzeugen wir selbst die Stundenlöhne
- D.h. wir spielen CEO (Head of HR)
- So kennen wir den wahren Zusammenhang und können später prüfen, ob man uns auf die Schliche kommt
- **Welche Stellen im Code sind unklar?**
- **Wo finden Sie die Gleichung (w)?**

First five wages are set as:

```
[[5.8899994]  
[5.62      ]  
[4.87      ]  
[6.5199995]  
[6.3999996]]
```

Mean of error term e: -0.001

Maximum of error term e: 0.663

```
#behave as if we were filling the world = we excute a data generating process  
school = df_WAGES['SCHOOL']  
school[2] #offset is zero, hence 3rd row and column SCHOOL  
exper = df_WAGES['EXPER']  
male = df_WAGES['MALE']  
  
#scikit needs matrix of size n_samples, n_features (van der Plas, p 349)  
print(school.shape)  
a_school = np.array(school, dtype='float32')  
a_school = a_school.reshape(school.size,1)  
print(a_school.shape)  
a_exper = np.array(exper, dtype='float32')  
a_exper = a_exper.reshape(exper.size,1)  
a_male = np.array(male, dtype='float32')  
a_male = a_male.reshape(male.size,1)  
print(a_male.shape)  
  
#head of HR determines wages according to the following TOP SECRET FORMULA  
a_wage_gen = -3.38 + 0.63 * a_school + 0.12 * a_exper + 1.34 * a_male  
print("First five wages are set as: \n", a_wage_gen[0:5,:])  
  
#to hide that discrimination happens, a bit of noise is added to this equation  
np.random.seed(2023)#make it reproducible  
a_eps = np.random.normal(loc=0.0, scale=0.2, size=school.size)  
a_eps = a_eps.reshape(school.size,1)  
print("Mean of error term e:", round(a_eps.mean(),3))  
print("Maximum of error term e:", round(a_eps.max(),3))  
a_wage_gen = a_wage_gen + a_eps
```

Experiment

- Die Gleichung (w) lautet also:
 $-3.38 + 0.63 \cdot \text{school} + 0.12 \cdot \text{exper} + 1.34 \cdot \text{male}$

Wir nutzen scikit um eine Gerade mit dem KQ Lernverfahren zu schätzen

- Wurde die wahre Gleichung exakt erkannt?**
- Wieviel Prozent der Varianz der Löhne wird erklärt?**
- Ändern Sie den seed auf Ihr Geburtsjahr. `np.random.seed(2023)`: Was kommt nun raus? Zufall?**

```
#####  
#   LinReg Scikit  
#####  
  
wages_linreg_sk = LinearRegression().fit(a_X, a_wage_gen)  
# Inspect the results  
print("Estimated paras are")  
print(wages_linreg_sk.intercept_)  
print(wages_linreg_sk.coef_)  
print("\n")  
print("REG: R squared on data")  
print(round(r2_score(a_wage_gen, wages_linreg_sk.predict(a_X)),3))
```

```
Estimated paras are  
[-3.3390803]  
[[0.626954  0.11917218 1.3396626  ]]
```

```
REG: R squared on data  
0.973
```

Tafel: Histogramm der
Schätzwerte für a & b

Theorem von Gauss & Markov

Wenn für die n Störterme e_i die folgenden Gauss-Markov Bedingungen gelten (Verbeek, 2012, 15-17):

- (a1) $E[e_i]=0, i=1,\dots,n$ (Im Mittel heben sich die Störungen weg, ignorierte Faktoren heben sich ggs auf)
- (a2) e und x sind unabhängig (Exogenität von x : Nur y durch Modell erklärt, x exogen)
- (a3) $Var[e_i]=\sigma^2$ (Homoskedastizität)
- (a4) $Cov[e_i, e_j]=0, i \neq j$ (Keine Autokorrelation im Störterm)
- (a5) Im Falle einer **multiplen** lin. Regression: Keine überflüssigen x !

Dann sind die **Schätzer α, β** die

- **Besten** (Sie haben die kleinste Varianz ...)
- **Linearen** (unter allen linearen ... Schätzern)
- **Unverzerrten Schätzer (Erwartungstreue)** ($E[\beta]=b$ und $E[\alpha]=a$, d.h. wir schätzen im Mittel richtig)

für die wahren Parameter a, b . Das nennt man auch die **BLUE Eigenschaft der KQ Schätzer α, β** .

Tafel: Schwerpunkt
der Schätzwerte &
Streubreite

Theorem von Gauss & Markov, Big Data Version

Wenn für die n Störterme e_i die folgenden Bedingungen gelten:

- (a1) $E[e_i]=0, i=1, \dots, n$ (Im Mittel heben sich Störungen weg, ignorierte Faktoren heben sich ggs auf)
- (a2) e und x sind unabhängig (Exogenität von x)
- (a3) $Var[e_i]$ variiert, d.h. Heteroskedastizität liegt vor oder Homoskedastizität
- (a4) Autokorrelation im Störterm geht nach „wenigen“ Lags auf Null
- (a5) Im Falle einer multiplen Regression: Keine lineare Abhängigkeit in x

Dann sind die **Schätzer α, β**

Tafel: Trichter um b

- **Konsistente** Schätzer (d.h. β konvergiert in WS gg b , analog α gg a ;
formal $\lim_{n \rightarrow \infty} P(|b - \beta_n| > d) = 0$ für $d > 0$ (beliebig klein!))
- Die t-Werte können auf Basis der Heteroscedasticity & Autocorrelation Consistent Standard Errors (**HAC SE**) als **normalverteilt** interpretiert werden

Anwendung

- Welche Stellen im Code sind unklar?
- Wurde die wahre Gleichung exakt erkannt?
- Gibt es Unterschiede zu scikit?

```
Estimated paras are  
[-3.3390803]  
[[0.626954  0.11917218 1.3396626  ]]
```

```
REG: R squared on data  
0.973
```

- Die t-Werte sind hier mit z überschrieben, weil wir nun über eine Normalverteilung sprechen

```
#####  
#   LinReg Statsmodels  
#####  
  
X = sm.add_constant(a_X)  
wages_linreg_sm = sm.OLS(a_wage_gen, X)#reverse order of args  
wages_linreg_sm_results = wages_linreg_sm.fit(cov_type='HAC',cov_kwds={'maxlags':10})  
#menu: 'HC3', 'HAC', etc  
wages_linreg_sm_results.params  
# Inspect the results  
print(wages_linreg_sm_results.summary())
```

```
===== OLS Regression Results =====  
Dep. Variable:          y      R-squared:          0.973  
Model:                OLS      Adj. R-squared:       0.973  
Method:             Least Squares  F-statistic:       3.931e+04  
Date:                Thu, 13 Jul 2023  Prob (F-statistic):    0.00  
Time:                17:14:27    Log-Likelihood:      678.56  
No. Observations:    3294      AIC:                -1349.  
Df Residuals:        3290      BIC:                -1325.  
Df Model:             3  
Covariance Type:      HAC  
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	-3.3391	0.030	-110.795	0.000	-3.398	-3.280
x1	0.6270	0.002	298.052	0.000	0.623	0.631
x2	0.1192	0.002	78.038	0.000	0.116	0.122
x3	1.3397	0.007	193.266	0.000	1.326	1.353

Anwendung

- Wir können für jeden der vier Modellparameter Hypothesentests durchführen
- Denn theoretisch könnte jeder Parameter ja auch eine Null sein
- Diese H0 wollen wir vier Mal überprüfen

Tafel: Normierung,
Glocke und 1,96
Sigma Intervall 95%

- **Wurde erkannt, dass hier diskriminiert wurde?**
- **Führen Sie die Regression auf den realen Löhnen aus!**
- -> PYTPRA_01_01_ScikitLinReg_SOLUTION

```
#####  
#   LinReg Statsmodels  
#####  
  
X = sm.add_constant(a_X)  
wages_linreg_sm = sm.OLS(a_wage_gen, X)#reverse order of args  
wages_linreg_sm_results = wages_linreg_sm.fit(cov_type='HAC',cov_kws={'maxlags':10})  
#menu: 'HC3', 'HAC', etc  
wages_linreg_sm_results.params  
# Inspect the results  
print(wages_linreg_sm_results.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	y	R-squared:	0.973			
Model:	OLS	Adj. R-squared:	0.973			
Method:	Least Squares	F-statistic:	3.931e+04			
Date:	Thu, 13 Jul 2023	Prob (F-statistic):	0.00			
Time:	17:14:27	Log-Likelihood:	678.56			
No. Observations:	3294	AIC:	-1349.			
Df Residuals:	3290	BIC:	-1325.			
Df Model:	3					
Covariance Type:	HAC					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-3.3391	0.030	-110.795	0.000	-3.398	-3.280
x1	0.6270	0.002	298.052	0.000	0.623	0.631
x2	0.1192	0.002	78.038	0.000	0.116	0.122
x3	1.3397	0.007	193.266	0.000	1.326	1.353